1. Problem Title & Source

Title: **Decrypt Message Using Key**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

A secret message has been encrypted using a **numeric key**.

You are given:

- A **string S** representing the encrypted message.
- An **integer K** representing the decryption key.

To decrypt the message, follow this rule:

For every character in the string:

- Shift the character **K positions backward in the alphabet**.
- If the shift goes before 'a', wrap around the alphabet.

This is similar to a **reverse Caesar cipher**.

Your task is to **decrypt the message and print the original string**.

3. Examples

Example 1

Input

d

3

Output

a

Explanation

d → shift back 3 positions

d → c → b → a

Result

a

Example 2

Input

khoor

3

Output

hello

Explanation

Each character shifts back by **3 positions**.

Encrypted	Decrypted
k	h
h	e
o	l
o	l
r	o

Result

hello

4. Constraints

1 ≤ length(S) ≤ 10^5

1 ≤ K ≤ 26

S contains lowercase letters only

Observations

- Alphabet size = **26**
- Wrap-around logic needed

5. Core Concept (Pattern)

This problem uses:

Character shifting

Modulo arithmetic

ASCII manipulation

It is essentially a **Caesar Cipher Decryption**.

6. Thought Process

Step 1 — Convert Character to Position

Example

d → position 3

(using 0-based indexing)

Step 2 — Shift Back by K

newPosition = (position - K + 26) % 26

Adding 26 prevents negative numbers.

Step 3 — Convert Back to Character

character = 'a' + newPosition

7. Visualization

Example

Encrypted = khoor

K = 3

Char	Position	New Position	Decrypted
k	10	7	h
h	7	4	e
o	14	11	l
o	14	11	l
r	17	14	o

Result

hello

8. Pseudocode

```
read string S
read integer K

result = ""

for each character c in S

    pos = c - 'a'

    newPos = (pos - K + 26) % 26

    newChar = 'a' + newPos

    append newChar to result

print result
```

9. Code Implementation

Python

```
s = input().strip()
k = int(input())
```

```
result = ""

for c in s:

    pos = ord(c) - ord('a')

    new_pos = (pos - k + 26) % 26

    result += chr(ord('a') + new_pos)

print(result)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        String s = sc.next();
        int k = sc.nextInt();

        StringBuilder result = new StringBuilder();

        for(char c : s.toCharArray()){

            int pos = c - 'a';

            int newPos = (pos - k + 26) % 26;

            char decrypted = (char)('a' + newPos);

            result.append(decrypted);
        }
```

```
        System.out.println(result);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(N)

Where

N = length of string

Space Complexity

O(N)

For storing decrypted message.

11. Common Mistakes

1. Forgetting wrap-around logic
2. Not adding **+26 before modulo**
3. Using wrong ASCII conversion
4. Incorrect input handling

12. Detailed Dry Run

Input

s = khoor

k = 3

Steps

Char	Position	New Position	Result
k	10	7	h
h	7	4	e
o	14	11	l
o	14	11	l
r	17	14	o

Output

hello

13. Real-Life Applications

Cipher techniques appear in:

- Secure communication
- Password hashing
- Encryption systems
- Cybersecurity algorithms

14. Common Traps

1. Confusing encryption and decryption
2. Using (pos - k) % 26 without adding 26
3. Incorrect character arithmetic
4. Ignoring wrap-around cases

15. Related Problems

Cipher / string manipulation:

LC 848 Shifting Letters

LC 1309 Decrypt String from Alphabet

LC 917 Reverse Only Letters

16. Alternate Approaches

Approach	Complexity	Notes
ASCII arithmetic	O(N)	optimal
Mapping table	O(N)	extra memory

17. Why This Solution Works

The alphabet is cyclic with **26 characters**, so shifting backwards with modulo ensures correct wrap-around.

Using modular arithmetic ensures characters stay within the alphabet range.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Encrypt message instead of decrypt
2. Handle **uppercase + lowercase**
3. Variable shift per character
4. Multi-key cipher system